

Fatec Ipiranga
Proposta de Aula: Árvores Binárias de Busca
Adelson-Velsky e Landis (AVL)

Disciplina: Estrutura de Dados (Prof. Veríssimo)

2026

1 Objetivo da Aula

Compreender a necessidade de balanceamento em Árvores Binárias de Busca (BST) e dominar a técnica de rotações para manter a complexidade de busca em $O(\log n)$.

2 O Problema: Degeneração de Árvores

Em uma BST comum, a ordem de inserção pode levar a uma estrutura linear (semelhante a uma lista encadeada). Se inserirmos os elementos $[1, 2, 3, 4]$, a altura h será igual a n , resultando em busca $O(n)$.

3 Definição de Árvore AVL

Uma árvore é dita **AVL** se, para cada nó, a diferença entre a altura da subárvore esquerda (h_e) e a altura da subárvore direita (h_d) é, no máximo, 1.

3.1 Fator de Balanceamento (FB)

O Fator de Balanceamento de um nó u é dado por:

$$FB(u) = h(u.esquerda) - h(u.direita) \quad (1)$$

Regra: Para ser AVL, $\forall u, FB(u) \in \{-1, 0, 1\}$.

4 Mergulho Profundo: Fator de Balanceamento (FB)

O segredo da AVL é que ela é uma árvore "autopoliciada". O FB é o sensor que dispara o alerta de manutenção.

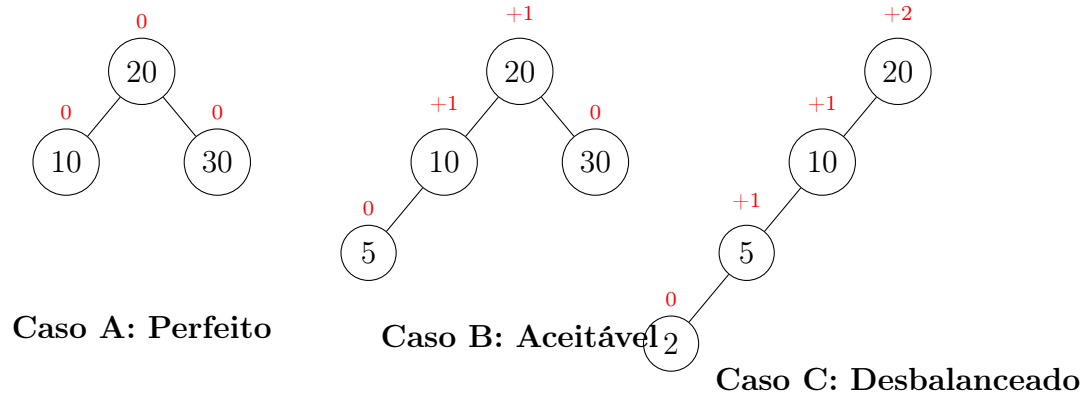
4.1 Cálculo da Altura

Para calcular o FB, definimos que a altura de um nó folha é 0, e a altura de um nó inexistente (NULL) é -1.

$$h(u) = 1 + \max(h(u.esq), h(u.dir)) \quad (2)$$

4.2 Exemplos Práticos de Cálculo

Abaixo, vemos como o FB se comporta em três cenários comuns. O valor dentro do nó é sua chave, e o valor acima (em vermelho) é o seu FB.



4.2.1 Análise dos Casos

- **Caso A:** $FB(20) = h(10) - h(30) \rightarrow 0 - 0 = 0$. Árvore ideal.
- **Caso B:** $FB(20) = h(10) - h(30) \rightarrow 1 - 0 = +1$. Ainda dentro do limite.
- **Caso C:** $FB(20) = h(10) - h(NULL) \rightarrow 2 - (-1) = +3$. **Erro crítico!**
(Nota: No exemplo visual simplificado, o nó 20 tem altura 3 à esquerda e -1 à direita, resultando em desequilíbrio severo).

4.2.2 Detalhamento do Cálculo: Por que a altura do nó (10) é 2?

É comum a confusão na contagem de nós com a altura. Vamos decompor o cálculo da altura e do FB para o **Caso C** de forma recursiva, de baixo para cima:

1. Nível das Folhas e NULLs:

- Por definição, qualquer nó inexistente (NULL) tem $h = -1$.
- O nó (2) é uma folha. Seus filhos são NULL.
- $h(2) = 1 + \max(h(NULL), h(NULL)) \rightarrow 1 + (-1) = 0$.

2. Subindo para o nó (5):

- O nó (5) tem o nó (2) à sua esquerda e NULL à sua direita.
- $h(5) = 1 + \max(h(2), h(NULL))$
- $h(5) = 1 + \max(0, -1) = 1 + 0 = 1$.

3. Chegando ao nó (10) [Onde surge o 2]:

- O nó (10) tem o nó (5) à sua esquerda e NULL à sua direita.
- $h(10) = 1 + \max(h(5), h(NULL))$
- $h(10) = 1 + \max(1, -1) = 1 + 1 = 2$.

4. Cálculo Final do FB no nó Crítico (20):

- O nó (20) tem o nó (10) à esquerda e NULL à direita.
- $FB(20) = h(10) - h(NULL)$
- $FB(20) = 2 - (-1) = 3$.

Nota Didática: Observe que a altura representa o número de "arestas" no caminho mais longo até uma folha. Se o nó (10) precisa descer duas arestas para chegar na folha mais profunda (o nó 2), sua altura é 2.

4.2.3 O Papel da Função $\max(h_e, h_d)$

A pergunta fundamental é: *"Se um nó tem dois filhos com alturas diferentes, qual deles define a altura do pai?"*

- **A Lógica da Altura:** A altura de um nó é o **caminho mais longo** dele até a folha mais profunda.
- **A Decisão:** Se o filho da esquerda tem altura 1 e o da direita é NULL (altura -1), o caminho mais longo passa, obrigatoriamente, pelo filho da esquerda.

No cálculo do nó **(10)**:

$$h(10) = 1 + \max(\underbrace{1}_{\text{filho esq}}, \underbrace{-1}_{\text{filho dir}}) = 1 + 1 = 2 \quad (3)$$

Por que não somar os dois? ou usar a média?

1. Se somássemos, estaríamos contando o número total de nós, não a altura (profundidade).
2. Ao usar o max, garantimos que a altura do pai seja sempre **1 unidade maior que a do seu filho mais alto**.

Analogia de Prédio: Imagine que um nó é o teto de um andar. Se do lado esquerdo o prédio tem 2 andares abaixo dele e do lado direito tem apenas um buraco (NULL), a altura total do teto em relação ao solo é definida pelo lado mais alto.

4.3 Tabela de Decisão de Rotação

Com base no sinal do FB do nó crítico e do seu filho, decidimos a rotação:

FB (Pai)	FB (Filho)	Direção do Peso	Ação Corretiva
+2	+1 ou 0	Esquerda-Esquerda (LL)	Rotação Simples Direita
-2	-1 ou 0	Direita-Direita (RR)	Rotação Simples Esquerda
+2	-1	Esquerda-Direita (LR)	Rotação Dupla Direita
-2	+1	Direita-Esquerda (RL)	Rotação Dupla Esquerda

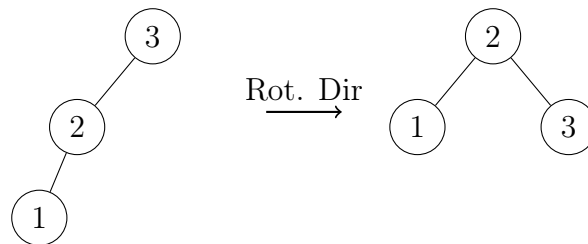
Table 1: Matriz de decisão para rebalanceamento AVL.

5 Operações de Balanceamento (Rotações)

Quando o $|FB| > 1$, realizamos rotações. Existem quatro casos principais:

5.1 Rotação Simples à Direita (LL)

Ocorre quando o desbalanceamento está "à esquerda da esquerda".



5.2 Rotação Dupla (LR e RL)

Quando o desbalanceamento é em "zigue-zague" (ex: esquerda da direita), primeiro fazemos uma rotação simples no filho para transformar em um caso de rotação simples no pai.

6 Pseudocódigo: Inserção

Algorithm 1 Inserção AVL (Lógica simplificada)

1. Realiza inserção padrão de BST.
 2. Atualiza a altura dos nós ancestrais.
 3. Calcula o FB de cada nó.
 4. **Se** $FB > 1$ ou $FB < -1$:
Executa a Rotação adequada (Simples ou Dupla).
-

7 Exercício Proposto

Desenhe o passo a passo da inserção dos seguintes valores em uma árvore AVL inicialmente vazia: **10, 20, 30, 25**. Identifique onde ocorre o desbalanceamento e qual rotação foi aplicada.